COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 4

The System Bus, Von Neuman Architecture & CPU Registers

At the end of the lesson, the student should be able to :

- Define Bus
- Identify types of Bus
- Identify the Memory Operations

MODULE 4.1

The Bus System

At the end of the lesson, the student should be able to :

- Define Bus
- Identify types of Bus
- Identify the Memory Operations

Buses

- Bus - A bus is a collection of wires on which electrical signals pass between components in the system.
- Data Bus - The 80x86 processors use the ***data bus to shuffle data between the various components in a computer system. The size of this bus varies widely in the 80x86 family.***

80x86 Processor	***Data Bus Size***
8088	8
80188	8
8086	16
80186	16
80286	16
80386sx	16
80386dx	32
80486	32
80586 / Pentium family	64

Address Bus

- Address Bus - The ***address bus specifies the location of data in memory. To differentiate memory locations and I/O devices, the system designer assigns a unique memory address to each memory element and I/O device***

- The number of bits on the address bus will determine the maximum number of addressable memory and I/O locations (2^n) n- bit binary number. The 8088 and 8086, for example, have 20 bit address busses. Therefore, they can access up to 1,048,576 memory locations.

80x86 Family Address Bus Sizes

Processor	***Address Bus*** Size	Max. Addressable	Memory In English
8088	20	1,048,576	1 Megabyte
8086	20	1,048,576	1 Megabyte
80188	20	1,048,576	1 Megabyte
80186	20	1,048,576	1 Megabyte
80286	24	16,777,216	16 Megabytes
80386sx	24	16,777,216	16 Megabytes
80386dx	32	4,294,976,296	4 Gigabytes
80486	32	4,294,976,296	4 Gigabytes
80586/Pentium	32	4,294,976,296	4 Gigabytes
Pentium Pro	36	68,719,476,736	64 Gigabytes
Pentium II	36	68,719,476,736	64 Gigabytes
Pentium III	36	68,719,476,736	64 Gigabytes

Control Bus

- **Control Bus** - The control bus is an extensive collection of signals that control how the processor communicates with the rest of the system.
- The ***read and write*** *control lines control the direction of data on the data bus. When both contain a logic one, the CPU and memory-I/O are not communicating with one another. If the read line is low (logic zero), the CPU is reading data from memory (that is, the system is transferring data from memory to the CPU). If the write line is low, the system transfers data from the CPU to memory.*
- The ***byte enable lines*** *are another set of important control lines. These control lines allow 16, 32, and 64 bit processors to deal with smaller chunks of data.*

Data Bus

The data bus is used to transfer data to and from the memory or I/O devices. It is bi-directional.

The address bus is used to select the desired memory or I/O devices by providing a unique address that corresponds to one of the memory or I/O. It is unidirectional.

The control bus is used to carry control signals to and from the memory or I/O devices. It is bi-directional.

Memory Operations

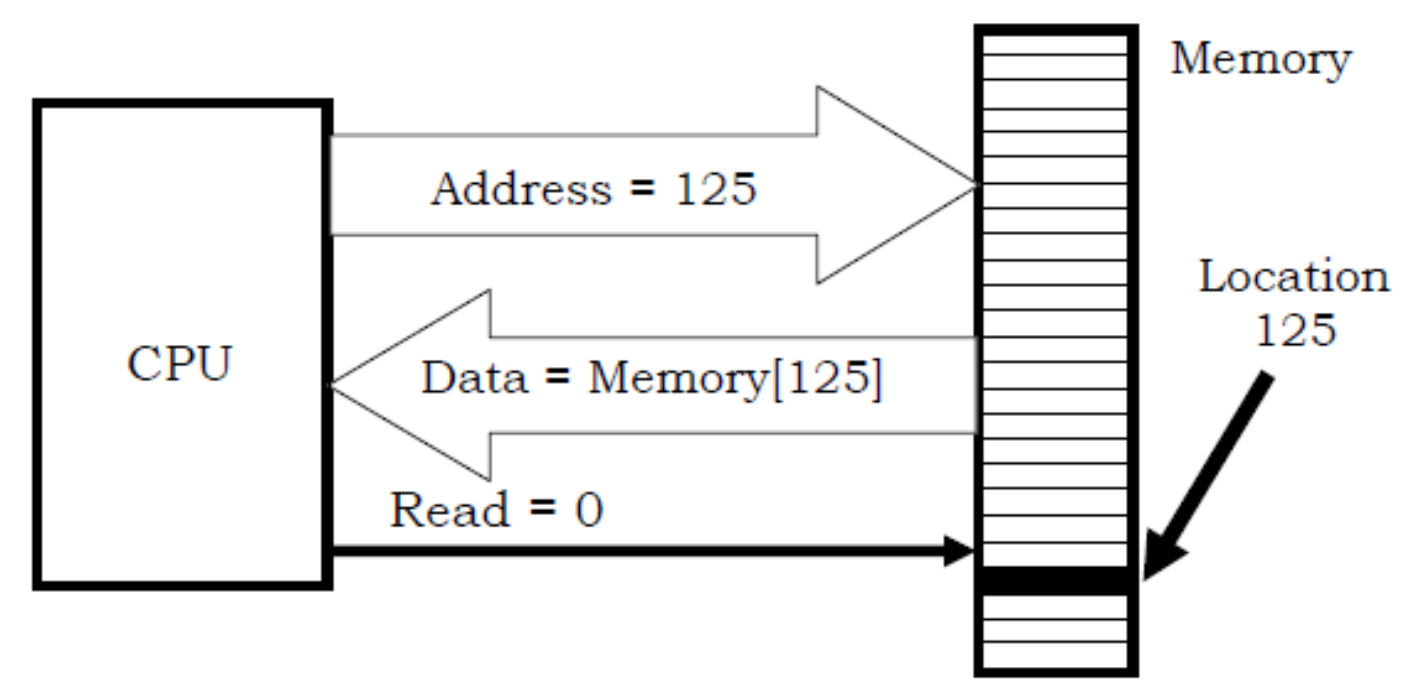
Fetch or Read. This transfers the contents of a specific MM location to the CPU. The word in the MM remains unchanged.

Read/Fetch Cycle:

1. CPU sends address of the desired location.
2. MM reads the data stored at that address and sends it to the CPU.

Reading the contents of Address 125.

The CPU places the address 125 on the address bus, enables the read line (since the CPU is reading data from memory), and then reads the resulting data from the data bus.



Memory Operations

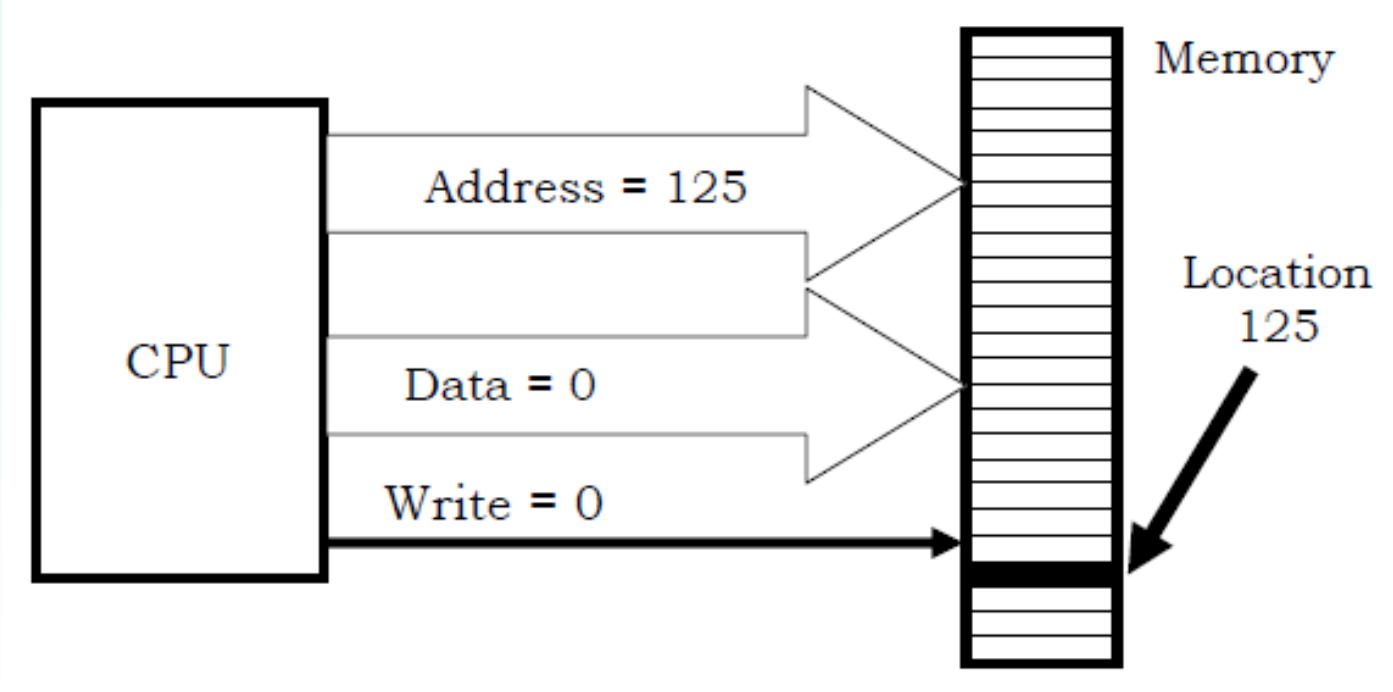
Store or Write. This transfers a word of information from the CPU to a specific MM location. This overwrites the former contents of that location.

Write/Store Cycle:

1. CPU sends address of the desired location to the MM, together with the data to be stored into that location.
2. Data is written at desired location.

Initializing the contents of address 125 to the value 0.

The CPU places the address 125 on the address bus, the value zero on the data bus, and enables the write line (since the CPU is writing data to memory).



Things to remember:

If operand or data needed by the instruction resides in MM:

1. It will have to be fetched by sending its address to the MAR and initiating a read cycle.

2. When the operand has been read from MM into the MDR, it may be transferred from the MDR to the ALU.

If result is to be stored in MM:

1. The result is sent to the MDR.

2. The address of the location where the result is to be stored is sent to the MAR and a write cycle is initiated

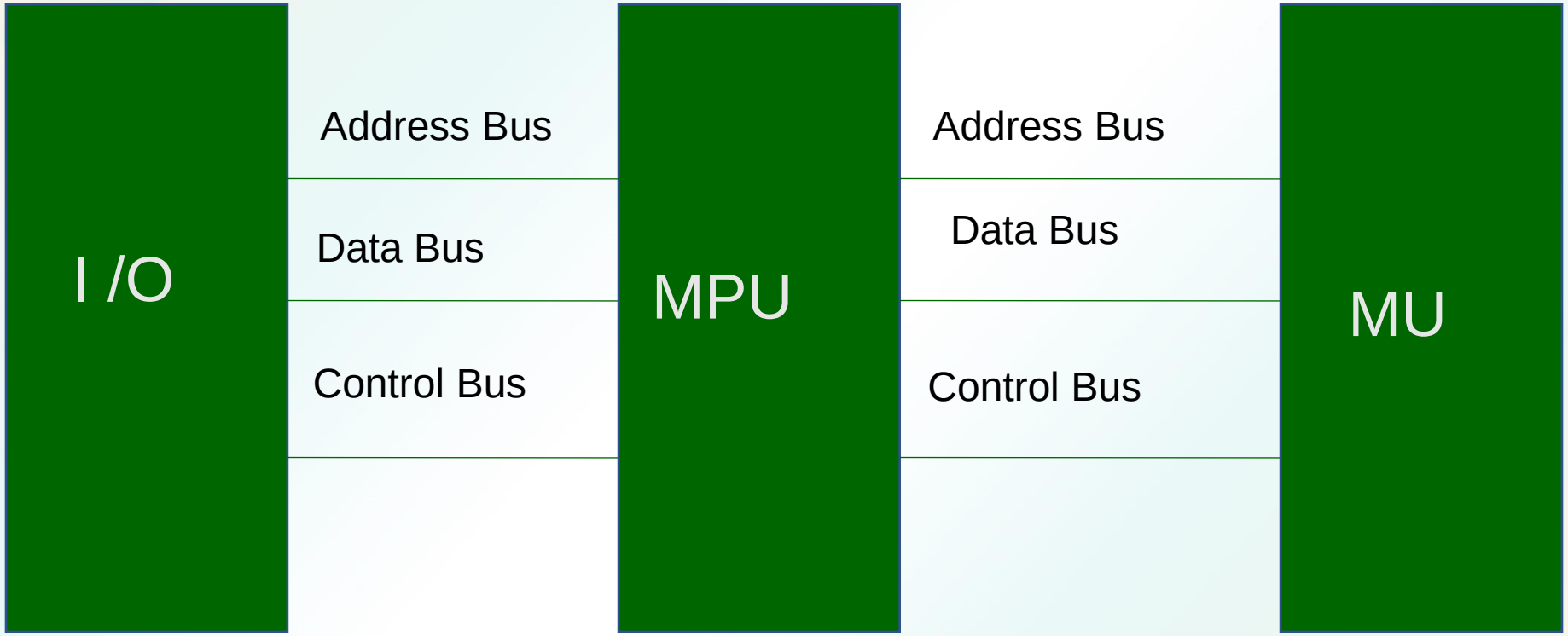
MODULE 4.1

Basic Operational Concept

At the end of the lesson, the student should be able to :

- Familiar with the Von Neumann Architecture
- Identify the functions of the different components of the Von Neumann Architecture
- Identify the operating steps of the Von Neumann Architecture

Von Neumann Architecture

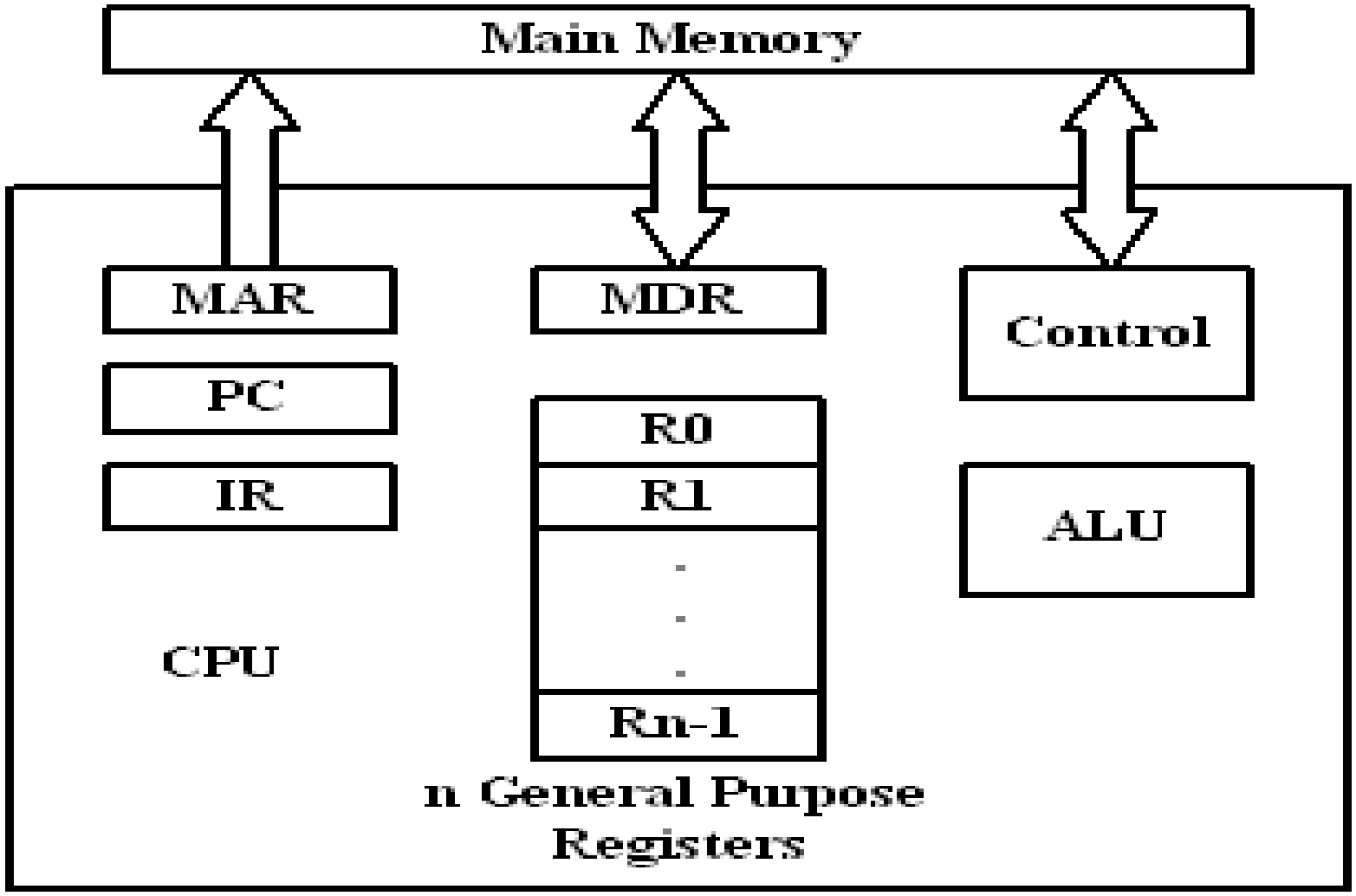


Von Neumann Architecture

Von Neumann Architecture means that programs (together with data) are stored in main memory during execution.

It is also known as the ***Stored Program Architecture*** or the ***Fetch-Decode-Execute Architecture***.

Connections between the processor and the main memory:



Functions of the Components

The PC (***Program Counter***) contains the memory address of the instruction to be executed. During execution, the contents of the PC are updated to point to the next instruction.

The MAR (***Memory Address Register***) holds the address of the location to or from which data are to be transferred.

The MDR (***Memory Data Register***) contains the data to be written or read out of the addressed location.

The IR (***Instruction Register***) contains the instruction that is being executed.

Operating Steps:

1. PC is set to point to the first instruction of the program (the operating system loads the memory address of the first instruction).
2. The contents of the PC are transferred to the MAR (which are automatically transmitted to the MM) and a *Read* signal is sent to the MM.
3. The addressed word is read out of MM and loaded into the MDR.
4. The contents of MDR are transferred to the IR. The instruction is ready to be decoded and executed.
5. During execution, the contents of the PC are incremented or updated to point to the next instruction.

Things to remember:

If operand or data needed by the instruction resides in MM:

1. It will have to be fetched by sending its address to the MAR and initiating a read cycle.

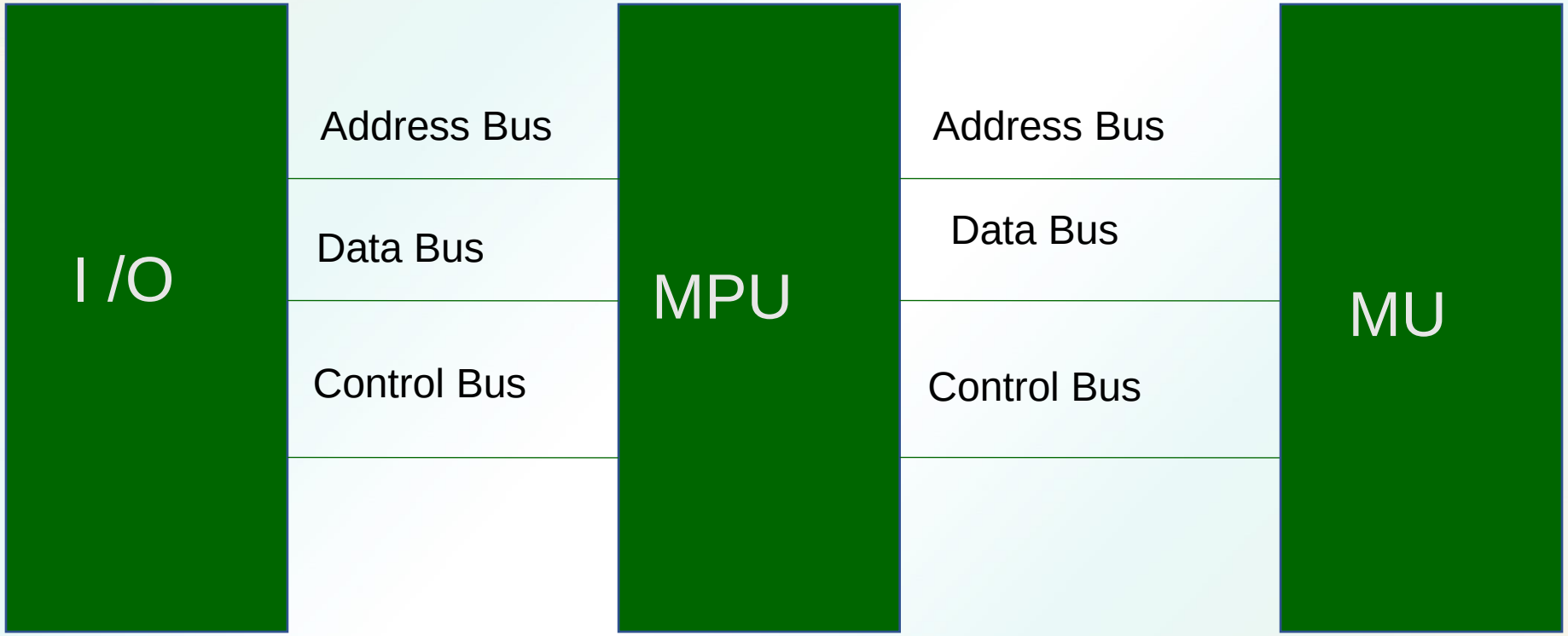
2. When the operand has been read from MM into the MDR, it may be transferred from the MDR to the ALU.

If result is to be stored in MM:

1. The result is sent to the MDR.

2. The address of the location where the result is to be stored is sent to the MAR and a write cycle is initiated

Von Neumann Architecture

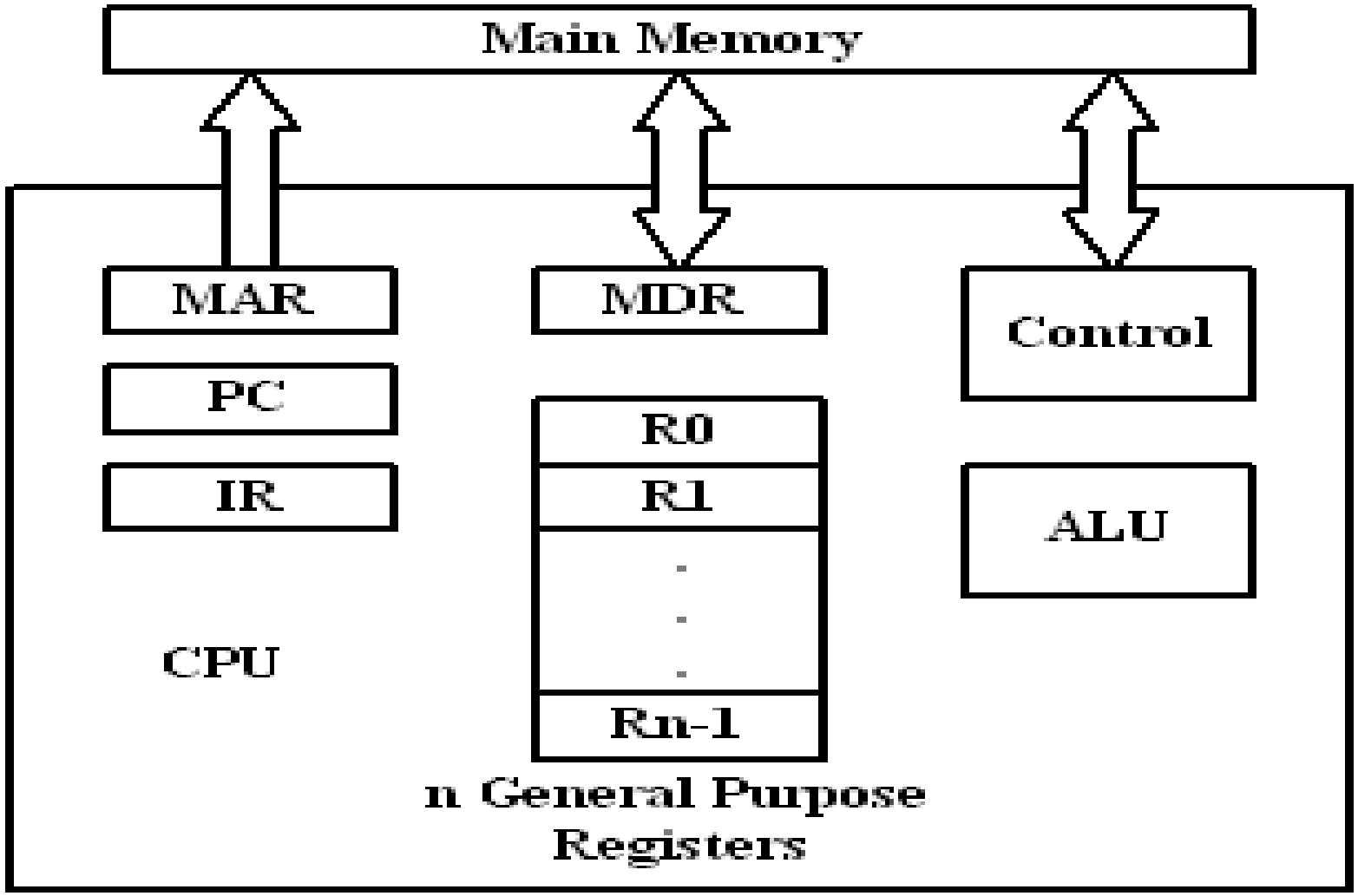


Von Neumann Architecture

Von Neumann Architecture means that programs (together with data) are stored in main memory during execution.

It is also known as the ***Stored Program Architecture*** or the ***Fetch-Decode-Execute Architecture***.

Connections between the processor and the main memory:



Functions of the Components

The PC (***Program Counter***) contains the memory address of the instruction to be executed. During execution, the contents of the PC are updated to point to the next instruction.

The MAR (***Memory Address Register***) holds the address of the location to or from which data are to be transferred.

The MDR (***Memory Data Register***) contains the data to be written or read out of the addressed location.

The IR (***Instruction Register***) contains the instruction that is being executed.

Operating Steps:

1. PC is set to point to the first instruction of the program (the operating system loads the memory address of the first instruction).
2. The contents of the PC are transferred to the MAR (which are automatically transmitted to the MM) and a *Read* signal is sent to the MM.
3. The addressed word is read out of MM and loaded into the MDR.
4. The contents of MDR are transferred to the IR. The instruction is ready to be decoded and executed.
5. During execution, the contents of the PC are incremented or updated to point to the next instruction.

Things to remember:

If operand or data needed by the instruction resides in MM:

1. It will have to be fetched by sending its address to the MAR and initiating a read cycle.

2. When the operand has been read from MM into the MDR, it may be transferred from the MDR to the ALU.

If result is to be stored in MM:

1. The result is sent to the MDR.

2. The address of the location where the result is to be stored is sent to the MAR and a write cycle is initiated

MODULE 4.3

CPU Registers and Flags

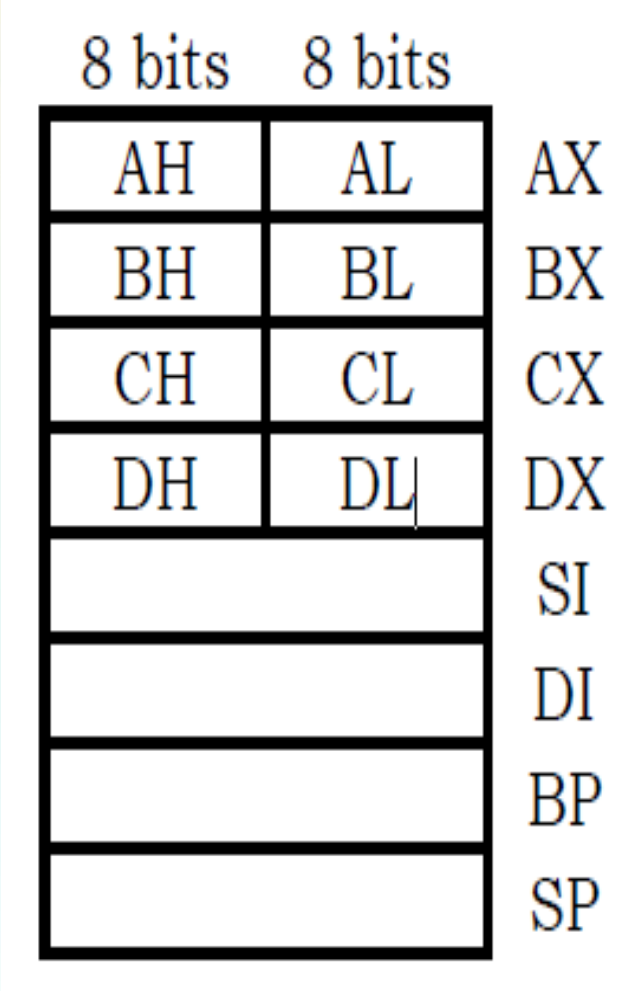
At the end of the lesson, the student should be able to :

- Define Register
- Identify the different types of Registers
- Identify the values of the conditional flag registers

Registers

Registers are high speed memory elements that resides inside the Central Processing Unit or CPU.

CPU General Purpose Registers



The **AX register (Accumulator) is where most arithmetic and logical computations take place.**

The **BX register (Base) has some special purposes as well. This is often used to hold base addresses of data located in the memory.**

The **CX register (Count), as its name implies, counts things. It is often use it to count off the number of iterations in a loop or specify the number of characters in a string.**

The **DX register (Data) has two special purposes: it holds the overflow from certain arithmetic operations, and it holds I/O addresses when accessing data on the 80x86 I/O bus.**

CPU's 8 bit registers

8 bits	8 bits	
AH	AL	AX
BH	BL	BX
CH	CL	CX
DH	DL	DX
		SI
		DI
		BP
		SP

AH
BH
CH
DH

Higher Order byte registers

AL
BL
CL
DL

Lower Order byte registers

Eight 16-bit registers

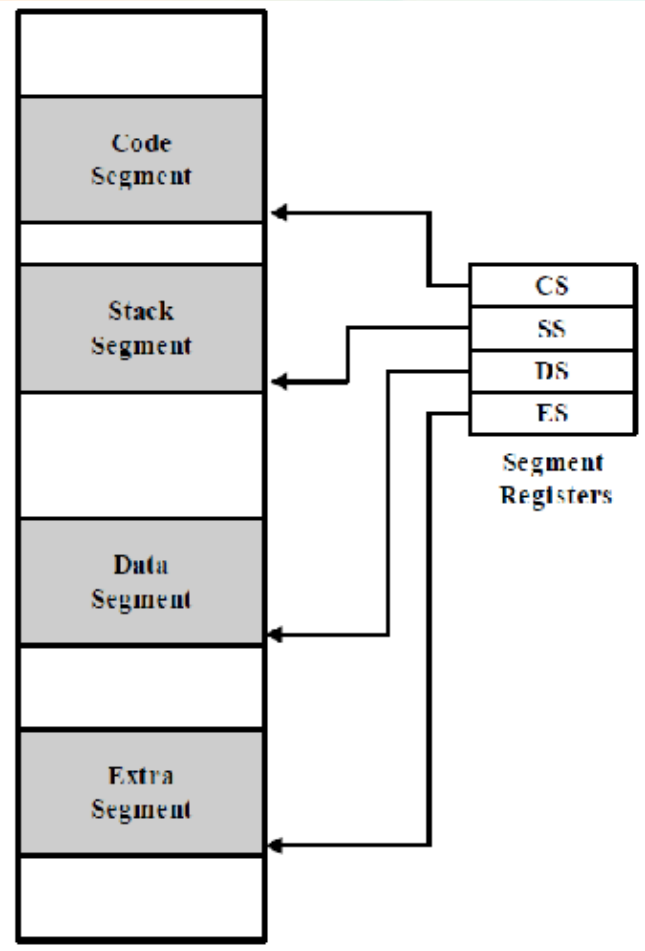
SEGMENT REGISTERS

Four Special Segment Registers

- 16 bits wide registers deals with selecting blocks (segments) of main memory. A segment register points at the beginning of a segment in memory.

- **CS (Code Segment)**
- **DS (Data Segment)**
- **ES (Extra Segment)**
- **SS (Stack Segment)**

Segment Registers



Segment Registers

- ***Code Segment.*** This contains the program or code. The address of the next instruction executed by the 8086/8088 is generated by adding the contents of IP (offset address) to the contents of CS x 10H.

Segment Registers

- ***Data Segment.*** This contains data referenced by almost all instructions and many addressing modes. Data are almost always moved into or out of MM via the data segment. The physical address of the data is generated by adding the contents of one of the index or pointer registers (BX, DI, or SI) to the contents of DS x 10H.

Segment Registers

- Stack Segment. This is for the LIFO stack. The physical address is a combination of the contents stack pointer (SP) plus SS x 10H.
- Extra Segment. This is normally for string instructions. When a string instruction is executed, the destination location is addressed by the destination index register (DI) plus ES x 10H.

Special Purpose Registers

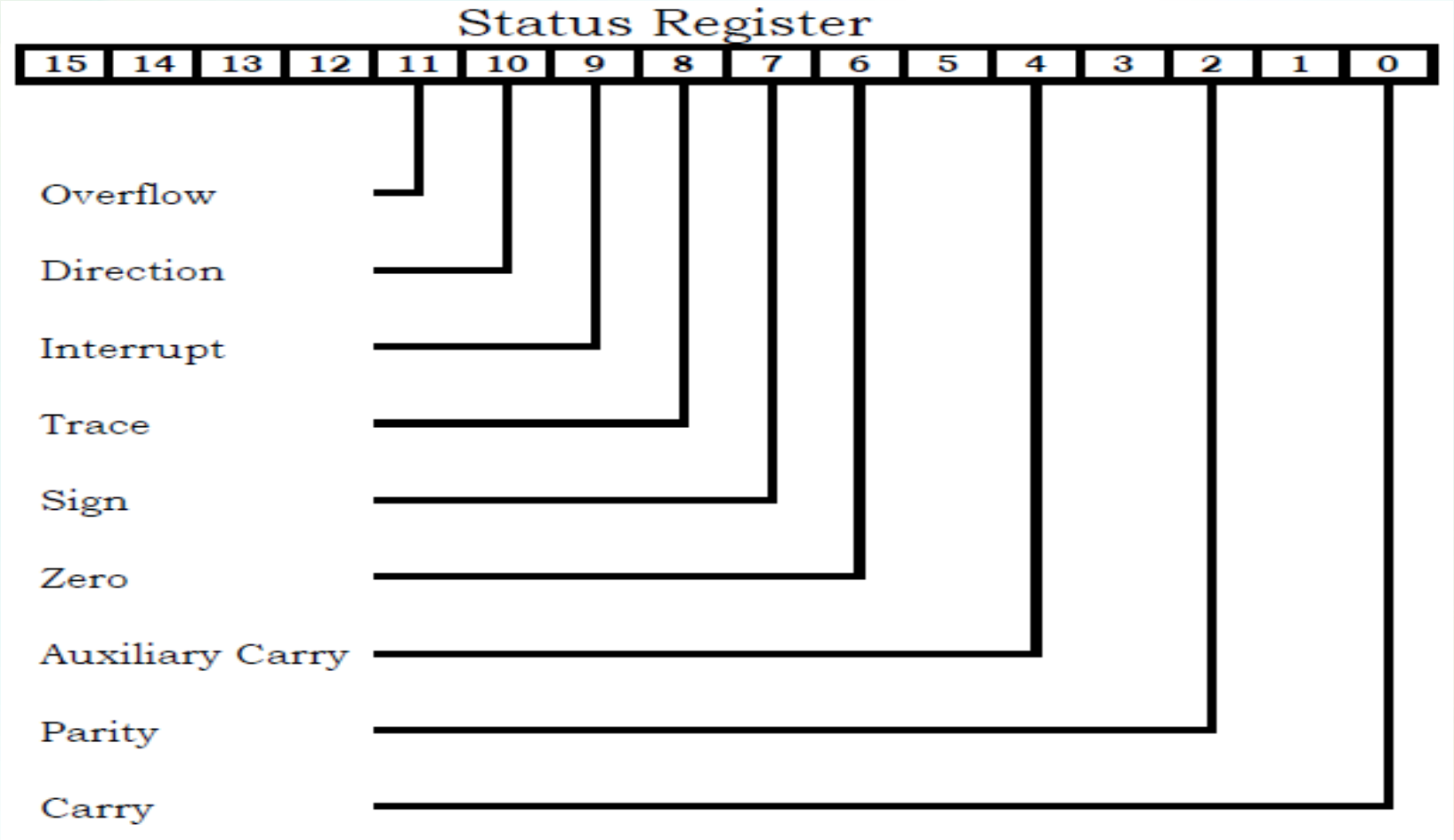
The instruction pointer (IP)

is also the program counter, it contains the memory address of the instruction to be executed. During execution, the contents of the IP are updated to point to the next instruction.

The flags register

is an extensive collection of one bit values which help determine the current state of the processor. A 16 bits wide register, it uses only nine of those bits.

Status Register



Each bit in the flags register is a flag

Conditional Flags - reflect the result of the previous operation involving the ALU.

Control Flags - (they control the execution of special functions.

The Conditional Flags

Sign Flag (SF) - b7

This is equal to the MSB of the result of the previous operation. 0 if positive, 1 if negative.

Zero Flag (ZF) - b6

This is set to 1 if the result of the previous operation is zero and 0 if the result is nonzero.

Parity Flag (PF) - b2

This is set to 1 if the low-order 8 bits of the result of the previous operation contain an even number of 1s. Otherwise it is reset to 0.

The Conditional Flags

Carry Flag (CF) - b0

An addition causes this flag to be set to 1 if there is a carry out of the MSB, and a subtraction causes it to be set to 1 if a borrow is needed.

Overflow Flag (OF) - b11

This is set to 1 if an overflow occurs, i.e., a result is out of range. More specifically, for addition this flag is set to 1 when there is a carry into the MSB and no carry out of the MSB or vice-versa. For subtraction, it is set to 1, when the MSB needs a borrow and there is no borrow from the MSB, or vice-versa.

The Conditional Flags

Auxiliary Carry Flag (AF) – b4

This flag is used exclusively for BCD arithmetic. It is set to 1 if there is a carry out of bit 3 (b3) during an addition or a borrow by bit 3 during a subtraction.

The Control Flags

Direction Flag (DF) - b10

This flag is used by string manipulation instructions. If clear, the string is processed from its beginning with the first element having the lowest address. Otherwise, the string is processed from the high address towards the low address.

Interrupt Enable Flag (IF) - b9

If set, a certain type of interrupt (a maskable interrupt) can be recognized by the CPU; otherwise, these interrupts are ignored.

The Control Flags

- Trap Flag (TF) - b8
  - If set, the 8086/8088 will enter into a single-step mode. In this mode, the CPU executes one instruction at a time.

80286 Registers

There are three additional bits present in the 80286 flags register. The I/O Privilege Level is a two bit value (bits 12 and 13). It specifies one of four different privilege levels necessary to perform I/O operations. These two bits generally contain 00 when operating in ***real mode*** on the 80286 (the 8086 emulation mode). The NT (***nested task***) flag controls the operation of an interrupt return (IRET) instruction. NT is normally zero for real-mode programs. The ***protected mode*** on the 80286 is to access more than one megabyte of RAM.

80286 Registers

80286 also has five additional registers used by an operating system to support memory management and multiple processes: the machine status word (***msw***), the global descriptor table register (***gdtr***), the local descriptor table register (***ldtr***), the interrupt descriptor table register (***idtr***) and the task register (***tr***).

80386/80486 Registers

The 80386 calls these new 32-bit versions **EAX**, **EBX**, **ECX**, **EDX**, **ESI**, **EDI**, **EBP**, **ESP**, **EFLAGS**, and **EIP** to differentiate them from their 16-bit versions (which are still available on the 80386).

Besides the 32-bit registers, the 80386 also provides two new 16-bit segment registers, **FS** and **GS**, which allow the programmer to concurrently access six different segments in memory.

80386/80486 Registers

The 80386 did not make any changes to the bits in the flags register. Instead, it extended the flags register to 32 bits (the “EFLAGS” register) and defined bits 16 and 17.

Bit 16 is the debug resume flag (**RF**) used with the set of 80386 debug registers.

Bit 17 is the Virtual 8086 mode flag (**VM**) which determines whether the processor is operating in virtual-86 mode (which simulates an 8086) or standard protected mode.

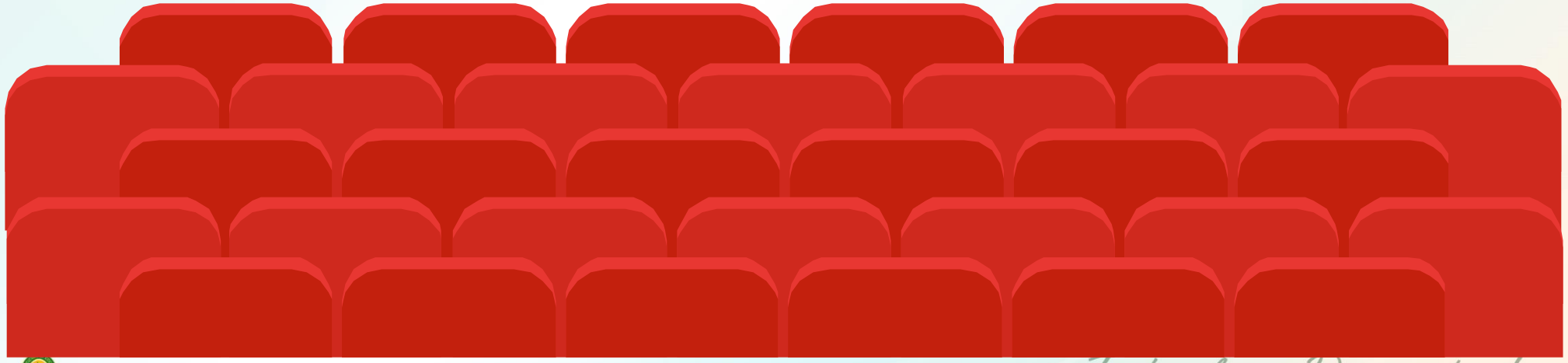
80386/80486 Registers

The 80486 adds a third bit to the EFLAGS register at bit 18 – the alignment check flag (**ACF**). Along with control register zero (**CR0**) on the 80486, this flag forces a trap (program abort) whenever the processor accesses non-aligned data.

The 80386 added four control registers: **CR0-CR3**. These registers extend the msw register of the 80286. On the 80386 and 80486 these registers control functions such as paged memory management, cache enable /disable operation (80486 only), protected mode operation, and more.

.

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU